BLOGGING PLATFORM

Project Documentation

A Java Full Stack Web Application

React.js | Spring Boot | MySQL

Objective of the Project

To design and develop a secure, full-stack blogging platform that allows users to register, authenticate, and manage blog posts through a modern web interface backed by a robust REST API and relational database.

1. Introduction

1.1 What is a Blogging Platform?

A blogging platform is a web application that allows users to create, publish, edit, and share written content, known as blog posts, with other users over the internet. It typically provides features such as user registration, secure login, content creation tools, and the ability to view and manage published articles.

1.2 Why This Project is Useful

This project demonstrates how modern web technologies can be combined to build a complete, production-style application. It gives learners hands-on experience with frontend development, backend REST API design, database management, and authentication - all essential skills for a Full Stack Developer. It also serves as a strong portfolio project that can be extended with advanced features.

1.3 Users of the System

- Admin - Manages users and has oversight of all blog content on the platform.
- Registered User - Can create an account, log in securely, and create, edit, view, and delete their own blog posts.

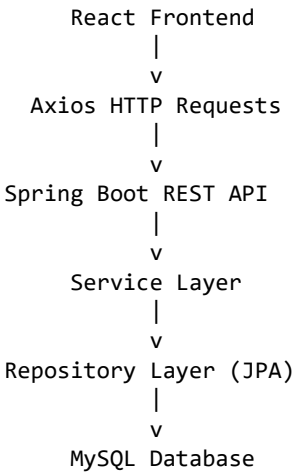
2. Technologies Used

Technology	Purpose
React.js	Building the dynamic frontend user interface
HTML5 / CSS3	Structuring and styling web pages
JavaScript	Client-side logic and interactivity
Axios	Sending HTTP requests from React to the backend API
React Router	Handling navigation between frontend pages
Java 21	Core programming language for the backend
Spring Boot	Building the REST API and backend application
Spring Data JPA	Simplifying database operations using ORM
Spring Security + JWT	Securing endpoints and handling authentication
Maven	Managing backend project dependencies and build
MySQL	Storing application data (users, posts, comments)
IntelliJ IDEA	Backend development environment
Visual Studio Code	Frontend development environment
MySQL Workbench	Designing and managing the database

3. Project Architecture

The application follows a layered client-server architecture. The React frontend runs in the user's browser and communicates with the Spring Boot backend through REST APIs over HTTP. The backend processes requests through its own internal layers before reading from or writing to the MySQL database.

3.1 Data Flow Diagram



3.2 Layer Responsibilities

- React Frontend: Renders the UI and captures user actions such as form submissions.
- Axios: Sends HTTP requests (GET, POST, PUT, DELETE) from React to the backend and returns responses.
- REST API (Controller): Receives HTTP requests and routes them to the correct service method.
- Service Layer: Contains the business logic - validation, processing, and coordination.
- Repository Layer (JPA): Converts Java objects into SQL queries and communicates with the database.
- MySQL Database: Stores and retrieves persistent data such as users, posts, and comments.

4. Project Development Process

The project is built in the following sequential steps:

1. Create a Spring Boot project using Spring Initializr.
2. Add dependencies: Spring Web, Spring Data JPA, MySQL Driver, Spring Security, JWT, Lombok.
3. Configure database connection details in application.properties.
4. Create the database schema in MySQL.
5. Create entity classes: User, Post, Comment.
6. Create repository interfaces for database access.
7. Create service classes containing business logic.
8. Create REST controllers to expose API endpoints.
9. Test backend APIs using Postman.
10. Create the React project.
11. Install required packages: axios and react-router-dom.
12. Create components: Login, Register, Home, Create Blog, Edit Blog, Blog Details, Navbar.
13. Connect React with Spring Boot using Axios.
14. Run the backend application.
15. Run the frontend application.

5. Backend and Frontend Connection

React and Spring Boot communicate through REST APIs using JSON as the data exchange format. When a user interacts with the browser, the request travels through several layers before a response is returned.

```
Browser
   |
   v
React Component
   |
   v
Axios Request
   |
   v
REST Controller
   |
   v
Service
   |
   v
Repository
   |
   v
MySQL
```

5.1 HTTP Methods Used

Method	Purpose
GET	Retrieve data, such as fetching the list of blog posts
POST	Create new data, such as registering a user or adding a post
PUT	Update existing data, such as editing a blog post
DELETE	Remove data, such as deleting a blog post

5.2 JSON Request and Response

Data is exchanged between React and Spring Boot in JSON format. When the frontend sends a request, such as creating a post, it sends a JSON object containing fields like title and content. The backend processes this data and returns a JSON response, either the created resource or a status message, which React then uses to update the UI.

6. Database Design

6.1 Users Table

Column	Description
id	Unique identifier for each user (Primary Key)
username	Unique username of the user
email	Email address of the user
password	Encrypted password used for authentication

6.2 Posts Table

Column	Description
id	Unique identifier for each post (Primary Key)
title	Title of the blog post
content	Main body/content of the blog post
createdDate	Date and time the post was created
userId	References the user who created the post (Foreign Key)

6.3 Comments Table

Column	Description
id	Unique identifier for each comment (Primary Key)
comment	Text content of the comment
postId	References the related post (Foreign Key)
userId	References the user who wrote the comment (Foreign Key)

7. Application Workflow

The end-to-end working of the application follows this sequence:

```
User Registration
      |
      v
Login using JWT
      |
      v
Dashboard
      |
      v
Create Blog
      |
      v
Store Blog in MySQL
      |
      v
Display Blogs
      |
      v
View Blog Details
      |
      v
Edit Blog
      |
      v
Delete Blog
      |
      v
Logout
```

8. Features

- User Registration
- Login Authentication
- JWT-based Security
- Create Blog Posts
- Read / View Blog Posts
- Update Blog Posts
- Delete Blog Posts
- Responsive User Interface
- RESTful APIs
- Persistent MySQL Storage

9. Advantages

- Provides hands-on experience with the complete full-stack development lifecycle.
- Uses industry-standard technologies (React, Spring Boot, MySQL) valued by employers.
- Secure authentication using JWT protects user data and API endpoints.
- Layered architecture improves code organization, readability, and maintainability.
- REST API design allows the frontend and backend to evolve independently.
- Scalable foundation that can be extended with new features or deployed to the cloud.

10. Future Enhancements

- Image Upload for blog posts
- Rich Text Editor for content creation
- Blog Categories and Tags
- Likes on blog posts
- Comments section on posts
- Search functionality
- Email Notifications
- Admin Dashboard for content and user management

11. Conclusion

The Blogging Platform project demonstrates how React, Spring Boot, and MySQL work together to build a secure, scalable, and maintainable full-stack web application. React delivers a fast and interactive user interface, Spring Boot exposes secure REST APIs protected by JWT authentication, and MySQL provides reliable, persistent data storage. Together, these technologies form a solid foundation for real-world web applications and serve as an excellent, extensible portfolio project for aspiring full-stack developers.